

# РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

## ОТЧЕТ

### ПО ЛАБОРАТОРНОЙ РАБОТЕ № 9

дисциплина:     Архитектура компьютера

Студент: Савенкова Татьяна Александровна

Группа: НКАбд-05-25

МОСКВА

2025 г.

## Оглавление

|                                                                |    |
|----------------------------------------------------------------|----|
| 1. Цель работы.....                                            | 4  |
| 2. Задание.....                                                | 5  |
| 3. Теоретическое введение.....                                 | 6  |
| 4. Выполнение лабораторной работы.....                         | 7  |
| <b>4.1 Реализация подпрограмм в NASM</b> .....                 | 7  |
| <b>4.1.1 Отладка программ с помощью GDB</b> .....              | 10 |
| <b>4.1.2 Добавление точек останова</b> .....                   | 14 |
| <b>4.1.3 Работа с данными программы в GDB</b> .....            | 14 |
| <b>4.1.4 Обработка аргументов командной строки в GDB</b> ..... | 19 |
| <b>4.2 Задание для самостоятельной работы</b> .....            | 20 |
| 5. Выводы .....                                                | 27 |
| 6. Список литературы.....                                      | 28 |

## Список иллюстраций

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Рисунок 1 Создание рабочего каталога .....                         | 7  |
| Рисунок 2 Код из листинга .....                                    | 7  |
| Рисунок 3 Запуск программы из листинга.....                        | 8  |
| Рисунок 4 Изменение программы первого листинга.....                | 8  |
| Рисунок 5 Результат.....                                           | 10 |
| Рисунок 6 Код из листинга 2 .....                                  | 11 |
| Рисунок 7 Запуск программы в отладчике.....                        | 11 |
| Рисунок 8 Проверка программы отладчиком .....                      | 12 |
| Рисунок 9 Запуск отладчика с брейкпойнтом .....                    | 12 |
| Рисунок 10 Дисассимилирование программы.....                       | 13 |
| Рисунок 11 Режим псевдографики.....                                | 13 |
| Рисунок 12 Список брейкпойнтов .....                               | 14 |
| Рисунок 13 Просмотр содержимого регистров .....                    | 15 |
| Рисунок 14 Просмотр содержимого переменных двумя способами .....   | 16 |
| Рисунок 15 Изменение содержимого переменных двумя способами.....   | 17 |
| Рисунок 16 Просмотр значения регистра разными представлениями..... | 18 |
| Рисунок 17 Примеры использования команды set .....                 | 19 |
| Рисунок 18 Подготовка новой программы.....                         | 19 |
| Рисунок 19 Проверка работы стека .....                             | 20 |
| Рисунок 20 Измененная программа предыдущей лабораторной работы ... | 21 |
| Рисунок 21 Код из листинга 3 .....                                 | 24 |
| Рисунок 22 Поиск ошибки в программе через пошаговую отладку .....  | 24 |
| Рисунок 23 Исправленный код.....                                   | 25 |
| Рисунок 24 Проверка корректировок в программе .....                | 25 |

## **1. Цель работы**

Приобретение навыков написания программ с использованием подпрограмм.

Знакомство с методами отладки при помощи GDB и его основными возможностями.

## **2. Задание**

1. Реализация подпрограмм в NASM
2. Отладка программ с помощью GDB
3. Самостоятельное выполнение заданий по материалам лабораторной работы

### 3. Теоретическое введение

Отладка — это процесс поиска и исправления ошибок в программе. В общем случае его можно разделить на четыре этапа: • обнаружение ошибки; • поиск её местонахождения; • определение причины ошибки; • исправление ошибки. Можно выделить следующие типы ошибок: • синтаксические ошибки — обнаруживаются во время трансляции исходного кода и вызваны нарушением ожидаемой формы или структуры языка; • семантические ошибки — являются логическими и приводят к тому, что программа запускается, отрабатывает, но не даёт желаемого результата; • ошибки в процессе выполнения — не обнаруживаются при трансляции и вызывают прерывание выполнения программы (например, это ошибки, связанные с переполнением или делением на ноль). Второй этап — поиск местонахождения ошибки. Некоторые ошибки обнаружить довольно трудно. Лучший способ найти место в программе, где находится ошибка, это разбить программу на части и произвести их отладку отдельно друг от друга. Третий этап — выяснение причины ошибки. После определения местонахождения ошибки обычно проще определить причину неправильной работы программы. Последний этап — исправление ошибки. После этого при повторном запуске программы, может обнаружиться следующая ошибка, и процесс отладки начнётся заново.

## 4. Выполнение лабораторной работы

### 4.1 Реализация подпрограмм в NASM

Создаю каталог для выполнения лабораторной работы №9 (Рисунок 1)

```
tasavenkova@fedora:~$ mkdir -p ~/work/arch-pc/lab09
tasavenkova@fedora:~$ cd ~/work/arch-pc/lab09
tasavenkova@fedora:~/work/arch-pc/lab09$ touch lab09-1.asm
tasavenkova@fedora:~/work/arch-pc/lab09$ ls
lab09-1.asm
tasavenkova@fedora:~/work/arch-pc/lab09$
```

Рисунок 1 Создание рабочего каталога

Копирую в файл код из листинга, компилирую и запускаю его, данная программа выполняет вычисление функции (Рисунок 2Рисунок 3).

```
GNU nano 8.3 /home/tasavenkova/work/arch-pc/lab09/lab09-1.asm
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '2x+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
;-----
; Основная программа
;-----
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax,x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax,result
call sprint
mov eax,[res]
call iprintLF
call quit
; Подпрограмма вычисления
; выражения "2x+7"
_calcul:
mov ebx,2
mul ebx
add eax,7
mov [res],eax
ret ; выход из подпрограммы
□
```

Рисунок 2 Код из листинга

```

tasavenkova@fedora:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
tasavenkova@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
tasavenkova@fedora:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 24
2x+7=55
tasavenkova@fedora:~/work/arch-pc/lab09$

```

Рисунок 3 Запуск программы из листинга

Изменяю текст программы, добавив в нее подпрограмму, теперь она вычисляет значение функции для выражения  $f(g(x))$  (Рисунок 4).

```

GNU nano 8.3 /home/tasavenkova/work/arch-pc/lab09/lab09-1
#include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB 'f(g(x))=2*(3x-1)+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
global _start
_start:
; Основная программа
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [res]
call iprintLF
call quit

; Подпрограмма вычисления f(g(x)) = 2*(3x-1)+7
_calcul:
push eax ; Сохраняем x в стеке
call _subcalcul ; Вызов подпрограммы _subcalcul (вычисляем g(x) = 3x-1)
; В eax теперь результат g(x)
mov ebx, 2 ; f(x) = 2x+7
mul ebx ; 2 * g(x)
add eax, 7 ; 2*g(x) + 7
mov [res], eax ; Сохраняем результат
pop eax ; Восстанавливаем регистр (не обязательно, но для чистоты)
ret ; выход из подпрограммы

; Подпрограмма вычисления g(x) = 3x-1
_subcalcul:
; В eax передано значение x

```

Рисунок 4 Изменение программы первого листинга

Код программы:

```

#include 'in_out.asm'

SECTION .data

msg: DB 'Введите x: ',0

```



```

result: DB 'f(g(x))=2*(3x-1)+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
global _start
_start:
; Основная программа
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [res]
call iprintLF
call quit

; Подпрограмма вычисления  $f(g(x)) = 2*(3x-1)+7$ 
_calcul:
push eax ; Сохраняем x в стеке
call _subcalcul ; Вызов подпрограммы _subcalcul (вычисляем  $g(x) = 3x-1$ )
; В eax теперь результат  $g(x)$ 
mov ebx, 2 ;  $f(x) = 2x+7$ 
mul ebx ;  $2 * g(x)$ 
add eax, 7 ;  $2*g(x) + 7$ 
mov [res], eax ; Сохраняем результат
pop eax ; Восстанавливаем регистр (не обязательно, но для чистоты)

```

ret ; выход из подпрограммы

; Подпрограмма вычисления  $g(x) = 3x-1$

\_subcalcul:

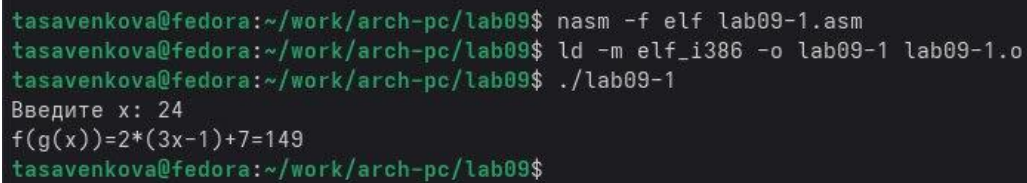
; В eax передано значение x

mov ebx, 3

mul ebx ;  $3 \cdot x$

sub eax, 1 ;  $3x-1$

ret ; возвращаем результат в eax



```
tasavenkova@fedora:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
tasavenkova@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
tasavenkova@fedora:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 24
f(g(x))=2*(3x-1)+7=149
tasavenkova@fedora:~/work/arch-pc/lab09$
```

Рисунок 5 Результат

#### 4.1.1 Отладка программ с помощью GDB

В созданный файл копирую программу второго листинга, транслирую с созданием файла листинга и отладки, компоную и запускаю в отладчике (Рисунок 6Рисунок 7).

```

GNU nano 8.3 /home/ta
SECTION .data
msg1: db "Hello, ",0x0
msg1Len: equ $ - msg1
msg2: db "world!",0xa
msg2Len: equ $ - msg2
SECTION .text
global _start
_start:
mov eax, 4
mov ebx, 1
mov ecx, msg1
mov edx, msg1Len
int 0x80
mov eax, 4
mov ebx, 1
mov ecx, msg2
mov edx, msg2Len
int 0x80
mov eax, 1
mov ebx, 0
int 0x80

```

Рисунок 6 Код из листинга 2

```

tasavenkova@fedora:~/work/arch-pc/lab09$ touch lab09-2.asm
tasavenkova@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-2.lst lab09-2.asm
tasavenkova@fedora:~/work/arch-pc/lab09$ d -m elf_i386 -o lab09-2 lab09-2.o
bash: d: команда не найдена...
tasavenkova@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-2 lab09-2.o
tasavenkova@fedora:~/work/arch-pc/lab09$ gdb lab09-2
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb)

```

Рисунок 7 Запуск программы в отладчике

Запустив программу командой `run`, я убедилась в том, что она работает исправно (Рисунок 8).

```
(gdb) run
Starting program: /home/tasavenkova/work/arch-pc/lab09/lab09-2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Download failed: Нет маршрута до узла. Continuing without separate debug info for system-supplied DS0 at 0xf7ffc000.
Hello, world!
[Inferior 1 (process 16519) exited normally]
(gdb)
```

**Рисунок 8 Проверка программы отладчиком**

Для более подробного анализа программы добавляю брейкпоинт на метку `start` и снова запускаю отладку (Рисунок 9)

```
[Inferior 1 (process 16519) exited normally]
(gdb) break _start
Breakpoint 1 at 0x8048000: file lab09-2.asm, line 9.
(gdb) run
Starting program: /home/tasavenkova/work/arch-pc/lab09/lab09-2
Download failed: Нет маршрута до узла. Continuing without separate debug info for system-supplied DS0 at 0xf7ffc000.

Breakpoint 1, _start () at lab09-2.asm:9
9      mov eax, 4
(gdb)
```

**Рисунок 9 Запуск отладчика с брейкпоинтом**

Далее смотрю дисассимилированный код программы, перевожу на команд с синтаксисом Intel amd топчик. Различия между синтаксисом АТТ и Intel заключаются в порядке операндов (АТТ - Операнд источника указан первым. Intel - Операнд назначения указан первым), их размере (АТТ - размер операндов указывается явно с помощью суффиксов, непосредственные операнды предваряются символом `$`; Intel - Размер операндов неявно определяется контекстом, как `ax`, `eax`, непосредственные операнды пишутся напрямую), именах регистров (АТТ - имена регистров предваряются символом `%`, Intel - имена регистров пишутся без префиксов)

```

(gdb) break _start
Breakpoint 1 at 0x8048080: file lab09-2.asm, line 9.
(gdb) run
Starting program: /home/tasavenkova/work/arch-pc/lab09/lab09-2
Download failed: Нет маршрута до узла. Continuing without separate debug info for system-supplied
t 0xf7ffc000.

Breakpoint 1, _start () at lab09-2.asm:9
9      mov eax, 4
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x8048080 <+0>:      mov     $0x4,%eax
0x8048085 <+5>:      mov     $0x1,%ebx
0x804808a <+10>:     mov     $0x8049000,%ecx
0x804808f <+15>:     mov     $0x0,%edx
0x8048094 <+20>:     int     $0x80
0x8048096 <+22>:     mov     $0x4,%eax
0x804809b <+27>:     mov     $0x1,%ebx
0x80480a0 <+32>:     mov     $0x8049000,%ecx
0x80480a5 <+37>:     mov     $0x7,%edx
0x80480aa <+42>:     int     $0x80
0x80480ac <+44>:     mov     $0x1,%eax
0x80480b1 <+49>:     mov     $0x0,%ebx
0x80480b6 <+54>:     int     $0x80
End of assembler dump.
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x8048080 <+0>:      mov     eax,0x4
0x8048085 <+5>:      mov     ebx,0x1
0x804808a <+10>:     mov     ecx,0x8049000
0x804808f <+15>:     mov     edx,0x0
0x8048094 <+20>:     int     0x80
0x8048096 <+22>:     mov     eax,0x4
0x804809b <+27>:     mov     ebx,0x1
0x80480a0 <+32>:     mov     ecx,0x8049000
0x80480a5 <+37>:     mov     edx,0x7
0x80480aa <+42>:     int     0x80
0x80480ac <+44>:     mov     eax,0x1
0x80480b1 <+49>:     mov     ebx,0x0
0x80480b6 <+54>:     int     0x80
End of assembler dump.
(gdb)

```

Рисунок 10 Дисассимилирование программы

Включаю режим псевдографики для более удобного анализа программы (Рисунок 11).

```

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcf50  0xffffcf50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080  0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B>0x8048080 <_start>  mov     eax,0x4
0x8048085 <_start+5>  mov     ebx,0x1
0x804808a <_start+10> mov     ecx,0x8049000
0x804808f <_start+15> mov     edx,0x0
0x8048094 <_start+20> int     0x80
0x8048096 <_start+22> mov     eax,0x4
0x804809b <_start+27> mov     ebx,0x1
0x80480a0 <_start+32> mov     ecx,0x8049000
0x80480a5 <_start+37> mov     edx,0x7
0x80480aa <_start+42> int     0x80
0x80480ac <_start+44> mov     eax,0x1
0x80480b1 <_start+49> mov     ebx,0x0
0x80480b6 <_start+54> int     0x80

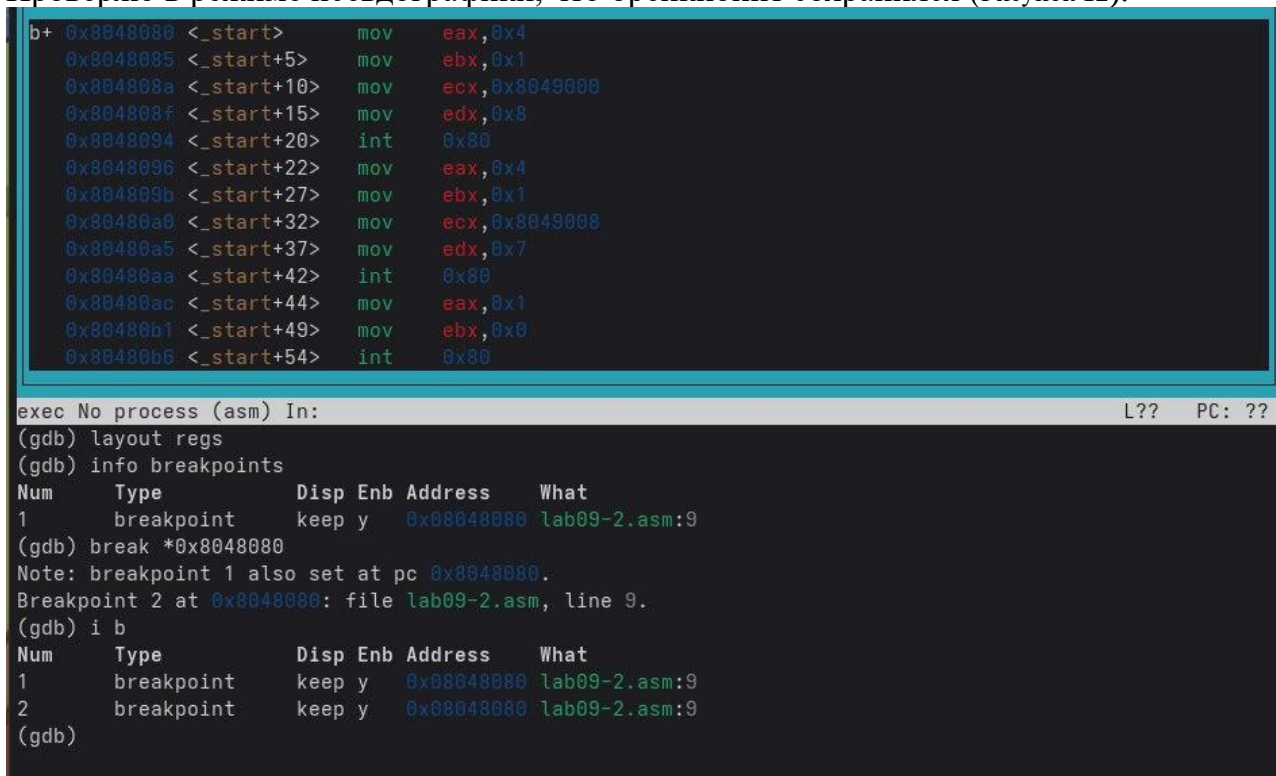
native process 16568 (asm) In: _start      L9      PC: 0x8048080
(gdb) layout regs
(gdb)

```

Рисунок 11 Режим псевдографики

### 4.1.2 Добавление точек останова

Проверяю в режиме псевдографики, что брейкпоинт сохранился (Рисунок 12).



```
b+ 0x8048080 <_start>    mov     eax,0x4
0x8048085 <_start+5>    mov     ebx,0x1
0x804808a <_start+10>   mov     ecx,0x8049000
0x804808f <_start+15>   mov     edx,0x8
0x8048094 <_start+20>   int     0x80
0x8048096 <_start+22>   mov     eax,0x4
0x804809b <_start+27>   mov     ebx,0x1
0x80480a0 <_start+32>   mov     ecx,0x8049000
0x80480a5 <_start+37>   mov     edx,0x7
0x80480aa <_start+42>   int     0x80
0x80480ac <_start+44>   mov     eax,0x1
0x80480b1 <_start+49>   mov     ebx,0x0
0x80480b6 <_start+54>   int     0x80

exec No process (asm) In:                                L??  PC: ??
(gdb) layout regs
(gdb) info breakpoints
Num      Type          Disp Enb Address      What
1        breakpoint    keep y  0x08048080 lab09-2.asm:9
(gdb) break *0x8048080
Note: breakpoint 1 also set at pc 0x8048080.
Breakpoint 2 at 0x8048080: file lab09-2.asm, line 9.
(gdb) i b
Num      Type          Disp Enb Address      What
1        breakpoint    keep y  0x08048080 lab09-2.asm:9
2        breakpoint    keep y  0x08048080 lab09-2.asm:9
(gdb)
```

Рисунок 12 Список брейкпоинтов

### 4.1.3 Работа с данными программы в GDB

Просматриваю содержимое регистров командой info registers (Рисунок 13)



```
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcf50 0xffffcf50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B> 0x8048080 <_start> mov eax,0x4
0x8048085 <_start+5> mov ebx,0x1
0x804808a <_start+10> mov ecx,0x8049000
0x804808f <_start+15> mov edx,0x8
0x8048094 <_start+20> int 0x80
0x8048096 <_start+22> mov eax,0x4
0x804809b <_start+27> mov ebx,0x1
0x80480a0 <_start+32> mov ecx,0x8049008
0x80480a5 <_start+37> mov edx,0x7
0x80480aa <_start+42> int 0x80
0x80480ac <_start+44> mov eax,0x1
0x80480b1 <_start+49> mov ebx,0x0
0x80480b6 <_start+54> int 0x80

native process 17041 (asm) In: _start L9 PC: 0x8048080
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcf50 0xffffcf50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
ds       0x2b     43
es       0x2b     43
--Type <RET> for more, q to quit, c to continue without paging--
```

Рисунок 13 Просмотр содержимого регистров

Смотрю содержимое переменных по имени и по адресу (Рисунок 14)

Register group: general

|        |            |                    |
|--------|------------|--------------------|
| eax    | 0x0        | 0                  |
| ecx    | 0x0        | 0                  |
| edx    | 0x0        | 0                  |
| ebx    | 0x0        | 0                  |
| esp    | 0xffffcf50 | 0xffffcf50         |
| ebp    | 0x0        | 0x0                |
| esi    | 0x0        | 0                  |
| edi    | 0x0        | 0                  |
| eip    | 0x8048080  | 0x8048080 <_start> |
| eflags | 0x202      | [ IF ]             |
| cs     | 0x23       | 35                 |
| ss     | 0x2b       | 43                 |

B+>0x8048080 <\_start>

```

mov    eax,0x4
0x8048085 <_start+5> mov    ebx,0x1
0x804808a <_start+10> mov    ecx,0x8049000
0x804808f <_start+15> mov    edx,0x8
0x8048094 <_start+20> int     0x80
0x8048096 <_start+22> mov    eax,0x4
0x804809b <_start+27> mov    ebx,0x1
0x80480a0 <_start+32> mov    ecx,0x8049008
0x80480a5 <_start+37> mov    edx,0x7
0x80480aa <_start+42> int     0x80
0x80480ac <_start+44> mov    eax,0x1
0x80480b1 <_start+49> mov    ebx,0x0
0x80480b6 <_start+54> int     0x80

```

native process 17041 (asm) In: \_start

L9 PC: 0x8048080

|        |           |                    |
|--------|-----------|--------------------|
| edi    | 0x0       | 0                  |
| eip    | 0x8048080 | 0x8048080 <_start> |
| eflags | 0x202     | [ IF ]             |
| cs     | 0x23      | 35                 |
| ss     | 0x2b      | 43                 |
| ds     | 0x2b      | 43                 |
| es     | 0x2b      | 43                 |

--Type <RET> for more, q to quit, c to continue without paging--

|    |     |   |
|----|-----|---|
| fs | 0x0 | 0 |
| gs | 0x0 | 0 |

```

(gdb) x/1sb &msg1
0x8049000 <msg1>:      "Hello, "
(gdb) x/1sb 0x8049008
0x8049008 <msg2>:      "world!\n\034"
(gdb)

```

Рисунок 14 Просмотр содержимого переменных двумя способами  
 Меняю содержимое переменных по имени и по адресу (Рисунок 15).



```

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcf50 0xffffcf50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B> 0x8048080 <_start> mov eax,0x4
0x8048085 <_start+5> mov ebx,0x1
0x804808a <_start+10> mov ecx,0x8049000
0x804808f <_start+15> mov edx,0x8
0x8048094 <_start+20> int 0x80
0x8048096 <_start+22> mov eax,0x4
0x804809b <_start+27> mov ebx,0x1
0x80480a0 <_start+32> mov ecx,0x8049008
0x80480a5 <_start+37> mov edx,0x7
0x80480aa <_start+42> int 0x80
0x80480ac <_start+44> mov eax,0x1
0x80480b1 <_start+49> mov ebx,0x0
0x80480b6 <_start+54> int 0x80

native process 17041 (asm) In: _start L9 PC: 0x8048080
(gdb) x/1sb &msg1
0x8049000 <msg1>: "Hello, "
(gdb) x/1sb 0x8049008
0x8049008 <msg2>: "world!\n\034"
(gdb) set {char}msg1='h'
'msg1' has unknown type; cast it to its declared type
(gdb) set {char}msg1='h'
No symbol "msg1" in current context.
(gdb) set {char}&msg1='h'
(gdb) x/1sb &msg1
0x8049000 <msg1>: "hello, "
(gdb) set {char}&msg2='x'
(gdb) x/1sb &msg2
0x8049008 <msg2>: "xorld!\n\034"
(gdb)

```

Рисунок 15 Изменение содержимого переменных двумя способами  
Вывожу в различных форматах значение регистра edx (Рисунок 16).

```

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x32      50
esp      0xffffcf50  0xffffcf50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080  0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B+>0x8048080 <_start> mov eax,0x4
0x8048085 <_start+5> mov ebx,0x1
0x804808a <_start+10> mov ecx,0x8049000
0x804808f <_start+15> mov edx,0x8
0x8048094 <_start+20> int 0x80
0x8048096 <_start+22> mov eax,0x4
0x804809b <_start+27> mov ebx,0x1
0x80480a0 <_start+32> mov ecx,0x8049008
0x80480a5 <_start+37> mov edx,0x7
0x80480aa <_start+42> int 0x80
0x80480ac <_start+44> mov eax,0x1
0x80480b1 <_start+49> mov ebx,0x0
0x80480b6 <_start+54> int 0x80

native process 17041 (asm) In: _start L9 PC: 0x8048080
No symbol "msg1" in current context.
(gdb) set {char}&msg1='h'
(gdb) x/1sb &msg1
0x8049000 <msg1>: "hello, "
(gdb) set {char}&msg2='x'
(gdb) x/1sb &msg2
0x8049008 <msg2>: "xorld!\n\034"
(gdb) set $ebx='2'
(gdb) p/s $ebx
$1 = 50
(gdb) p/t $ecx
$2 = 0
(gdb) p/x $edx
$3 = 0x0
(gdb)

```

Рисунок 16 Просмотр значения регистра разными представлениями

С помощью команды set меняю содержимое регистра ebx (Рисунок 17).

```

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x2      2
esp      0xffffcf50 0xffffcf50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B+>0x8048080 <_start> mov    eax,0x4
0x8048085 <_start+5> mov    ebx,0x1
0x804808a <_start+10> mov    ecx,0x8049000
0x804808f <_start+15> mov    edx,0x8
0x8048094 <_start+20> int    0x80
0x8048096 <_start+22> mov    eax,0x4
0x804809b <_start+27> mov    ebx,0x1
0x80480a0 <_start+32> mov    ecx,0x8049008
0x80480a5 <_start+37> mov    edx,0x7
0x80480aa <_start+42> int    0x80
0x80480ac <_start+44> mov    eax,0x1
0x80480b1 <_start+49> mov    ebx,0x0
0x80480b6 <_start+54> int    0x80

native process 17041 (asm) In: _start L9 PC: 0x8048080
(gdb) p/s $ebx
$1 = 50
(gdb) p/t $ecx
$2 = 0
(gdb) p/x $edx
$3 = 0x0
(gdb) set $ebx='2'
(gdb) p/s
$4 = 0
(gdb) p/s $ebx
$5 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$6 = 2
(gdb)

```

Рисунок 17 Примеры использования команды set

Разница в выводе команд p/s \$ebx до и после set \$ebx=2 объясняется форматом вывода и изменением значения регистра

#### 4.1.4 Обработка аргументов командной строки в GDB

Копирую программу из предыдущей лабораторной работы в текущий каталог и создаю исполняемый файл с файлом листинга и отладки (Рисунок 18).

```

(gdb) layout asm
tasavenkova@fedora:~/work/arch-pc/lab09$ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab09-3.asm
tasavenkova@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-3.lst lab09-3.asm
tasavenkova@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-3 lab09-3.o
tasavenkova@fedora:~/work/arch-pc/lab09$

```

Рисунок 18 Подготовка новой программы

Запускаю программу в режиме отладки с указанием аргументов, указываю брейкпоинт и запускаю отладку. Проверяю работу стека, изменяя аргумент команды просмотра регистра esp на +4, число обусловлено разрядностью системы, а указатель

void занимает как раз 4 байта, ошибка при аргументе +24 означает, что аргументы на вход программы закончились. (Рисунок 19).

```
tasavenkova@fedora:~/work/arch-pc/lab09$ gdb --args lab09-3 аргумент1 фргумент 2 'аргумент 3'
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb) b _start
Breakpoint 1 at 0x8048148: file lab09-3.asm, line 5.
(gdb) run
Starting program: /home/tasavenkova/work/arch-pc/lab09/lab09-3 аргумент1 фргумент 2 аргумент\ 3

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Download failed: Нет маршрута до узла. Continuing without separate debug info for system-supplied DSO at 0xf7ffc000.

Breakpoint 1, _start () at lab09-3.asm:5
5      pop ecx ; Извлекаем из стека в `ecx` количество
(gdb) x/x $esp
0xfffffcf10:  0x00000005
(gdb) x/s *(void**)(esp + 4)
0xffffd0f0:  "/home/tasavenkova/work/arch-pc/lab09/lab09-3"
(gdb) x/s *(void**)(esp + 8)
0xffffd11d:  "аргумент1"
(gdb) x/s *(void**)(esp + 12)
0xffffd12f:  "фргумент"
(gdb) x/s *(void**)(esp + 16)
0xffffd140:  "2"
(gdb) x/s *(void**)(esp + 20)
0xffffd142:  "аргумент 3"
(gdb) x/s *(void**)(esp + 24)
0x0:      <error: Cannot access memory at address 0x0>
(gdb) █
```

Рисунок 19 Проверка работы стека

#### 4.2 Задание для самостоятельной работы

1. Меняю программу самостоятельной части предыдущей лабораторной работы с использованием подпрограммы (Рисунок 20).

```

GNU nano 8.3 /home/tasavenkova/work/arch-pc/lab08/lab9-s

; Получаем количество аргументов
pop ecx ; argc
pop edx ; argv[0] - имя программы
sub ecx, 1 ; исключаем имя программы
mov esi, 0 ; общая сумма

; Проверяем, есть ли аргументы
cmp ecx, 0
jz _end

next:
; Берем следующий аргумент
pop eax
call atoi ; преобразуем строку в число

; ВЫЗОВ ПОДПРОГРАММЫ для вычисления  $f(x) = 5x + 17$ 
call _calcul

; Добавляем результат к общей сумме
add esi, eax

; Переходим к следующему аргументу
loop next

_end:
mov eax, msg_result
call sprint
mov eax, esi
call iprintLF
call quit

; ПОДПРОГРАММА вычисления  $f(x) = 5x + 17$ 
; Вход: eax = x
; Выход: eax =  $5x + 17$ 
_calcul:
push ebx ; Сохраняем ebx
mov ebx, 5 ; Коэффициент 5
mul ebx ;  $eax = 5 * x$ 
add eax, 17 ;  $eax = 5x + 17$ 
pop ebx ; Восстанавливаем ebx
ret ; Возврат из подпрограммы

```

Рисунок 20 Измененная программа предыдущей лабораторной работы  
Код программы:

```
%include 'in_out.asm'
```

SECTION .data

msg\_func db "Функция:  $f(x) = 17 + 5x$ ", 0

msg\_result db "Результат: ", 0

SECTION .text

global \_start

\_start:

mov eax, msg\_func

call sprintLF

; Получаем количество аргументов

pop ecx ; argc

pop edx ; argv[0] - имя программы

sub ecx, 1 ; исключаем имя программы

mov esi, 0 ; общая сумма

; Проверяем, есть ли аргументы

cmp ecx, 0

jz \_end

next:

; Берем следующий аргумент

pop eax

call atoi ; преобразуем строку в число

; ВЫЗОВ ПОДПРОГРАММЫ для вычисления  $f(x) = 5x + 17$

call \_calcul

; Добавляем результат к общей сумме

add esi, eax

; Переходим к следующему аргументу

loop next

\_end:

mov eax, msg\_result

call sprint

mov eax, esi

call iprintLF

call quit

; ПОДПРОГРАММА вычисления  $f(x) = 5x + 17$

; Вход: eax = x

; Выход: eax =  $5x + 17$

\_calcul:

push ebx ; Сохраняем ebx

mov ebx, 5 ; Коэффициент 5

mul ebx ; eax =  $5 * x$

add eax, 17 ; eax =  $5x + 17$

pop ebx ; Восстанавливаем ebx

ret ; Возврат из подпрограммы

2. Запускаю программу в режиме отладчика и пошагово через si просматриваю изменение значений регистров через i r. При выполнении инструкции mul esx можно заметить, что результат умножения записывается в регистр eax, но также меняет и edx. Значение регистра ebx не обновляется напрямую, поэтому результат программа неверно подсчитывает функцию (Рисунок 22).



```

GNU nano 8.3 /home/tasavenkova/work/arch-pc/lab09/lab9-
#include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov ebx,3
mov eax,2
add ebx,eax
mov ecx,4
mul ecx
add ebx,5
mov edi,ebx
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit

```

Рисунок 21 Код из листинга 3

```

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x3      3
esp      0xffffcf50 0xffffcf50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x804816d 0x804816d <_start+5>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
ds       0x2b     43

lab9-s.asm
1 %include 'in_out.asm'
2 SECTION .data
3 div: DB 'Результат: ',0
4 SECTION .text
5 GLOBAL _start
6 _start:
7 ; ---- Вычисление выражения (3+2)*4+5
8 mov ebx,3
9 mov eax,2
10 add ebx,eax
11 mov ecx,4
12 mul ecx
13 add ebx,5
14 mov edi,ebx

native process 18035 (src) In: _start L9 PC: 0x804816d
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x3      3
esp      0xffffcf50 0xffffcf50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x804816d 0x804816d <_start+5>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
ds       0x2b     43
es       0x2b     43
fs       0x0      0
--Type <RET> for more, q to quit, c to continue without paging--

```

Рисунок 22 Поиск ошибки в программе через пошаговую отладку  
Исправляю найденную ошибку. (Рисунок 23)



```
GNU nano 8.3 /home/tasavenkova/wo
#include 'in_out.asm'

SECTION .data
div: DB 'Результат: ',0

SECTION .text
global _start
_start:

; --- Вычисление выражения (3+2)*4+5
mov eax,3      ; eax = 3
mov ebx,2      ; ebx = 2
add eax,ebx    ; eax = 3+2 = 5
mov ebx,4      ; ebx = 4
mul ebx        ; eax = 5*4 = 20
add eax,5      ; eax = 20+5 = 25
mov edi,eax    ; сохраняем результат

; --- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF

call quit
```

Рисунок 23 Исправленный код

Теперь программа верно считает значение функции (Рисунок 24).

```
tasavenkova@fedora:~/work/arch-pc/lab09$ nasm -f elf lab9-s.asm
tasavenkova@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-s lab9-s.o
tasavenkova@fedora:~/work/arch-pc/lab09$ ./lab9-s
Результат: 25
tasavenkova@fedora:~/work/arch-pc/lab09$
```

Рисунок 24 Проверка корректировок в программе

Код измененной программы: %include 'in\_out.asm'

SECTION .data

div: DB 'Результат: ',0

SECTION .text

global \_start

\_start:

; --- Вычисление выражения  $(3+2)*4+5$

mov eax,3 ; eax = 3

mov ebx,2 ; ebx = 2

add eax,ebx ; eax = 3+2 = 5

mov ebx,4 ; ebx = 4

mul ebx ; eax = 5\*4 = 20

add eax,5 ; eax = 20+5 = 25

mov edi,eax ; сохраняем результат

; --- Вывод результата на экран

mov eax,div

call sprint

mov eax,edi

call iprintLF

call quit

## **5. Выводы**

В результате выполнения данной лабораторной работы я приобрела навыки написания программ с использованием подпрограмм, а также познакомилась с методами отладки при помощи GDB и его основными возможностями.

## 6. Список литературы

1. [https://esystem.rudn.ru/pluginfile.php/2089096/mod\\_resource/content/0/Лабораторная%20работа%20№9.%20Понятие%20подпрограммы.%20Отладчик%20..pdf](https://esystem.rudn.ru/pluginfile.php/2089096/mod_resource/content/0/Лабораторная%20работа%20№9.%20Понятие%20подпрограммы.%20Отладчик%20..pdf)
2. <https://esystem.rudn.ru/course/view.php?id=112>